pfSense Firewall Home Lab Project

Blocking SYN Flood Attacks with Custom Firewall Rules

Ronak - Cybersecurity Home Lab
April 2026

Project Overview

Simulated and mitigated a SYN flood (DoS) attack using pfSense. Analyzed abnormal traffic patterns (~30k packets/sec), identified service impact, and implemented optimized firewall rules to block malicious traffic while maintaining availability.

When I decided to build this lab, my goal was simple but important: I wanted real, hands-on experience with configuring and defending a production-grade firewall - the kind of work SOC analysts and network security engineers do every day.

I chose pfSense because it is a widely used open-source firewall that many companies rely on for perimeter defense. On my ASUS Windows laptop, I set up pfSense inside VirtualBox and created a realistic attack environment with two separate networks:

- WAN side (Attack side): Where the attacker (Kali Linux) would try to overwhelm the firewall.
- LAN side (Management side): Where I could safely access the pfSense web GUI and manage the firewall.

I simulated a real SYN flood attack using hping3 from Kali, watched how it affected the firewall, then created custom firewall rules to block the attack. Finally, I verified everything worked and reflected on how this mirrors real SOC operations.

This lab gave me practical experience with firewall installation, rule creation, attack simulation, log analysis, and troubleshooting.

Lab Environment & Network Topology

Host Machine: ASUS Windows laptop running VirtualBox

Firewall: pfSense virtual appliance

Attacker: Kali Linux VM on the WAN side (IP 10.0.2.15)

Management/Admin Machine: Ubuntu VM on the LAN side (IP 192.168.1.11)

Network Details:

The "Attack" Side (WAN)

- pfSense WAN Gateway: 10.0.2.3
- Kali Linux (Attacker): 10.0.2.15

The "Management" Side (LAN)

- pfSense LAN Gateway: 192.168.1.1
- Ubuntu Machine: 192.168.1.11

I used bridged networking in VirtualBox for both VMs so they could communicate directly with pfSense.

Step-by-Step Walkthrough

1. Installing pfSense and Initial Console Configuration

I downloaded the pfSense ISO and created a new VM in VirtualBox on my ASUS laptop. I allocated 2 CPUs, 4 GB RAM, and 32 GB disk. After booting, I completed the text-based console setup wizard (hostname, domain, WAN interface). I also reset the default admin password.

2. Web GUI Access and Basic Network Configuration

I logged into the pfSense web GUI using the LAN IP. I configured the WAN interface properly and enabled the DHCP server on the LAN side so the Ubuntu machine could get an IP automatically.

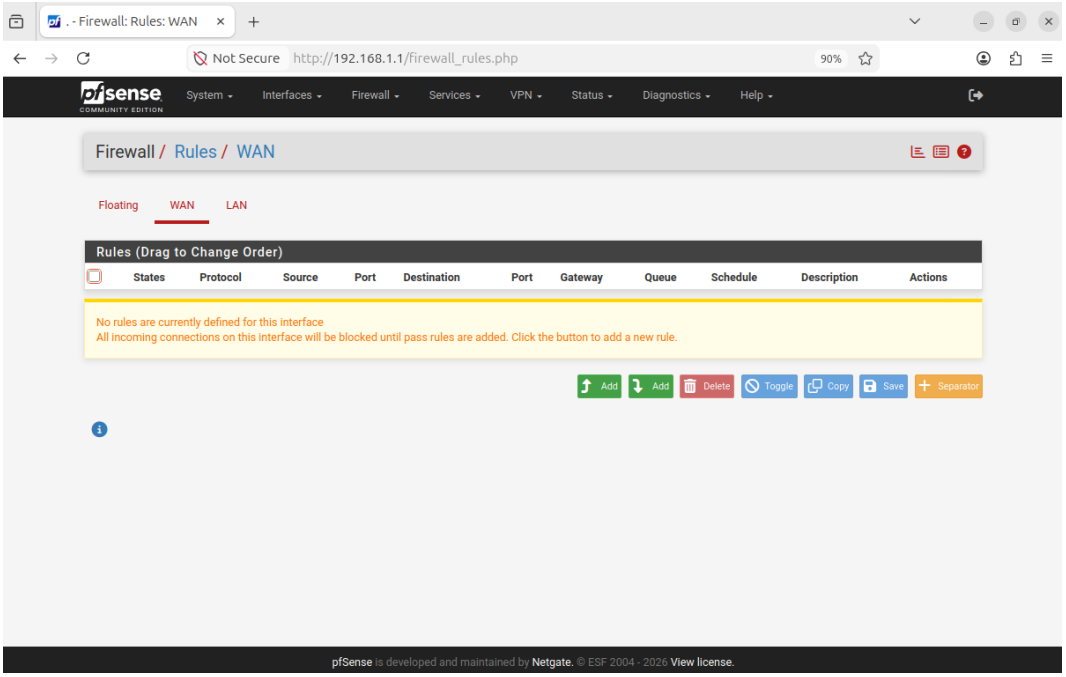


Figure 1: Configuring the WAN Interface (Gateway 10.0.2.3)

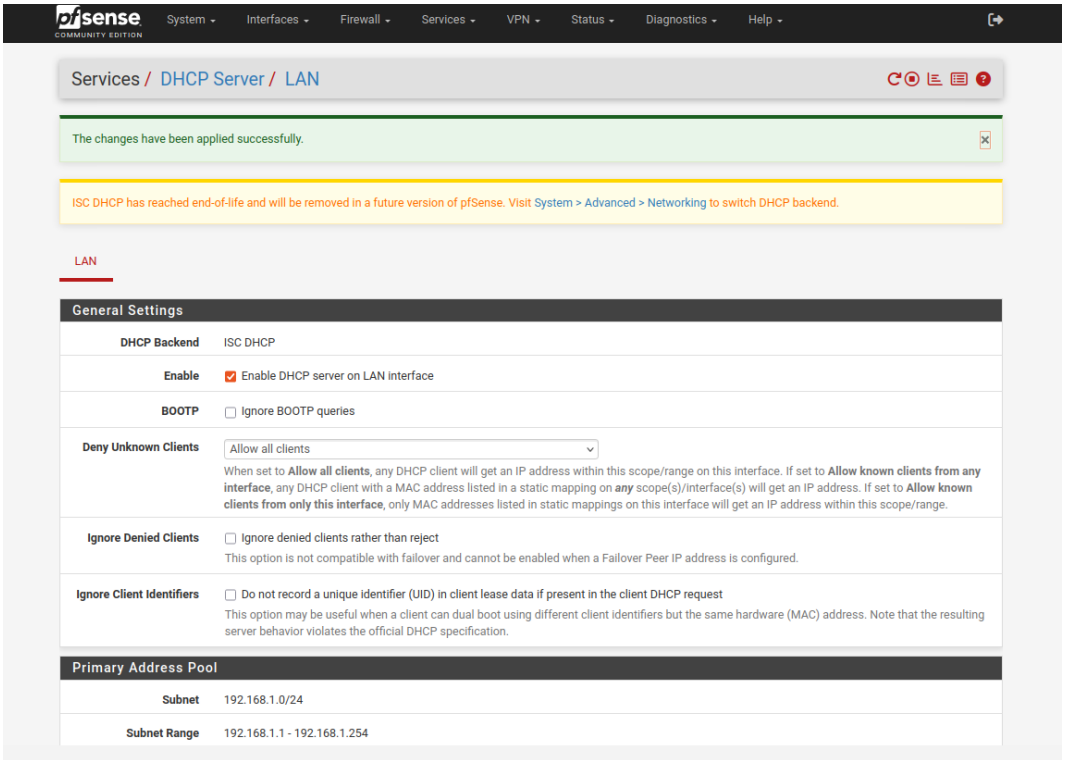
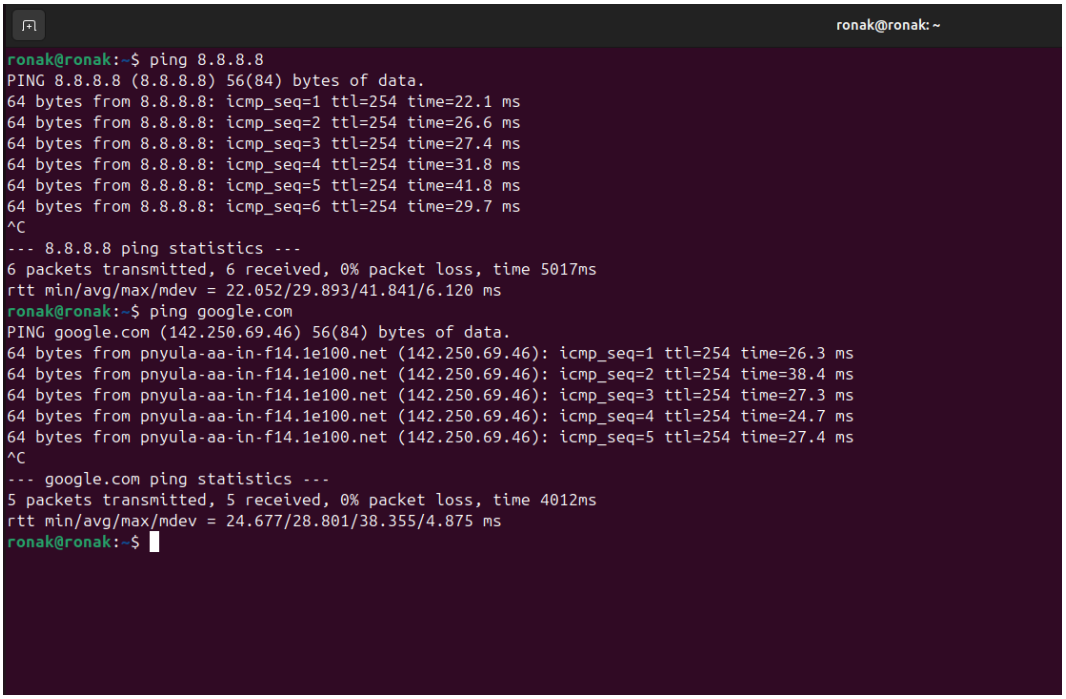


Figure 2: DHCP server configuration confirmed on LAN side



```
ronak@ronak: ~
ronak@ronak:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=254 time=22.1 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=254 time=26.6 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=254 time=27.4 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=254 time=31.8 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=254 time=41.8 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=254 time=29.7 ms
^C
--- 8.8.8.8 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5017ms
rtt min/avg/max/mdev = 22.052/29.893/41.841/6.120 ms
ronak@ronak:~$ ping google.com
PING google.com (142.250.69.46) 56(84) bytes of data.
64 bytes from pnyula-aa-in-f14.1e100.net (142.250.69.46): icmp_seq=1 ttl=254 time=26.3 ms
64 bytes from pnyula-aa-in-f14.1e100.net (142.250.69.46): icmp_seq=2 ttl=254 time=38.4 ms
64 bytes from pnyula-aa-in-f14.1e100.net (142.250.69.46): icmp_seq=3 ttl=254 time=27.3 ms
64 bytes from pnyula-aa-in-f14.1e100.net (142.250.69.46): icmp_seq=4 ttl=254 time=24.7 ms
64 bytes from pnyula-aa-in-f14.1e100.net (142.250.69.46): icmp_seq=5 ttl=254 time=27.4 ms
^C
--- google.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4012ms
rtt min/avg/max/mdev = 24.677/28.801/38.355/4.875 ms
ronak@ronak:~$
```

Figure 3: Successful ping from Kali Linux to pfSense (before rules)

Once the DHCP server was running and I confirmed connectivity with ping, I realized how critical proper interface and DHCP configuration is. A single mistake here could have left the entire lab unreachable or exposed. This step helped me understand how real network administrators ensure basic connectivity before moving on to security rules.

3. Creating Custom Firewall Rules

I went to Firewall → Rules and created two custom rules on the WAN interface: one to block the Kali attacker IP (10.0.2.15) and another to block SYN flood attempts. I enabled logging on both rules.

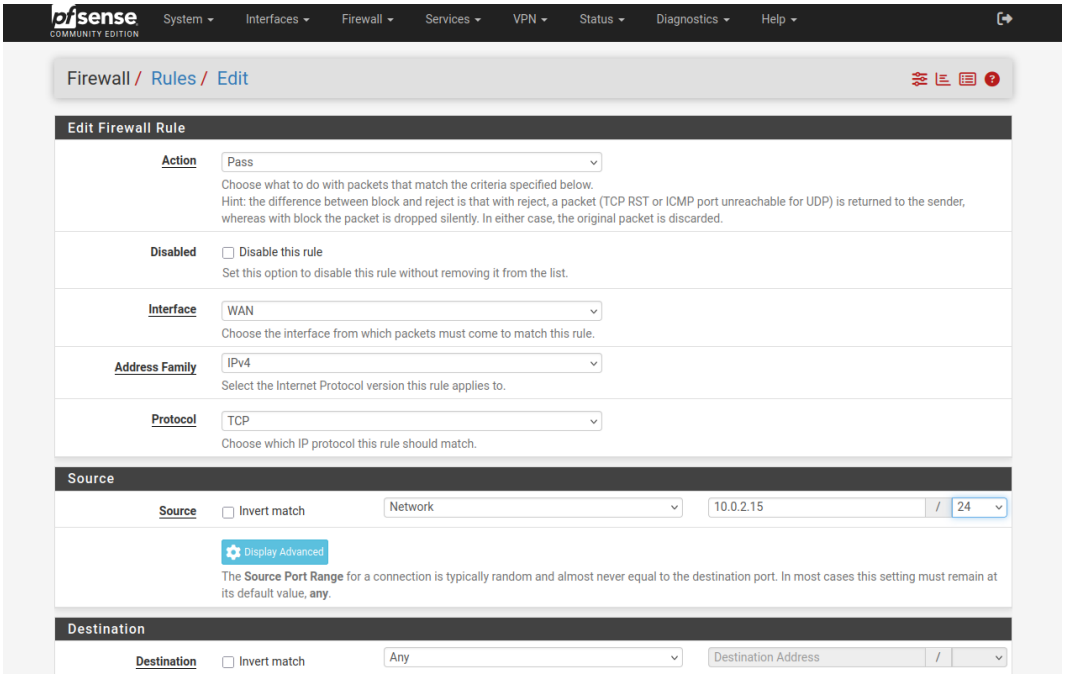


Figure 4: Creating custom firewall rule - blocking source IP

I deliberately created two separate rules instead of one combined rule. The first rule blocked the specific Kali attacker IP (10.0.2.15), which is a common technique used in SOCs to quickly stop targeted attacks. The second rule was designed to catch SYN flood behavior in general. I placed both rules at the top of the WAN interface list and enabled logging so I could see exactly which packets were being dropped. This gave me real insight into how rule order and logging work together in production firewalls.

I also made sure to test the rules in a controlled way, I knew that one wrong setting could accidentally block legitimate traffic, which is a common real-world mistake.

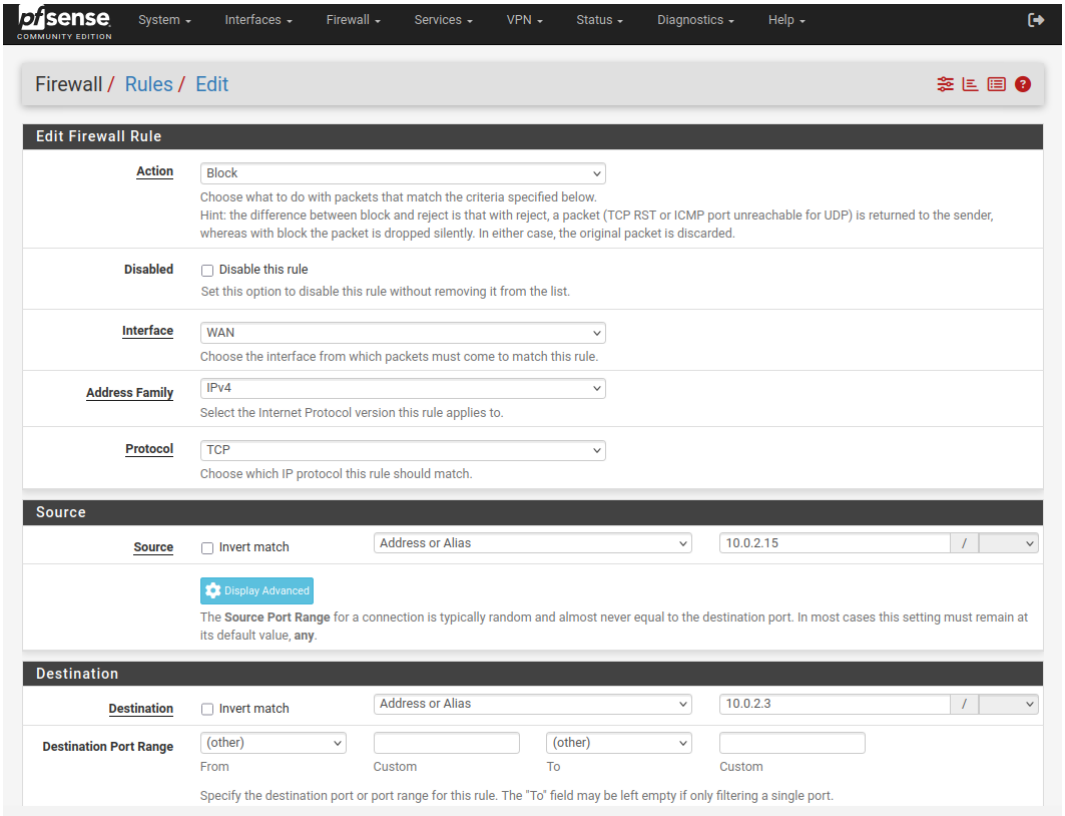


Figure 5: Setting block rule for hping3 SYN flood attack

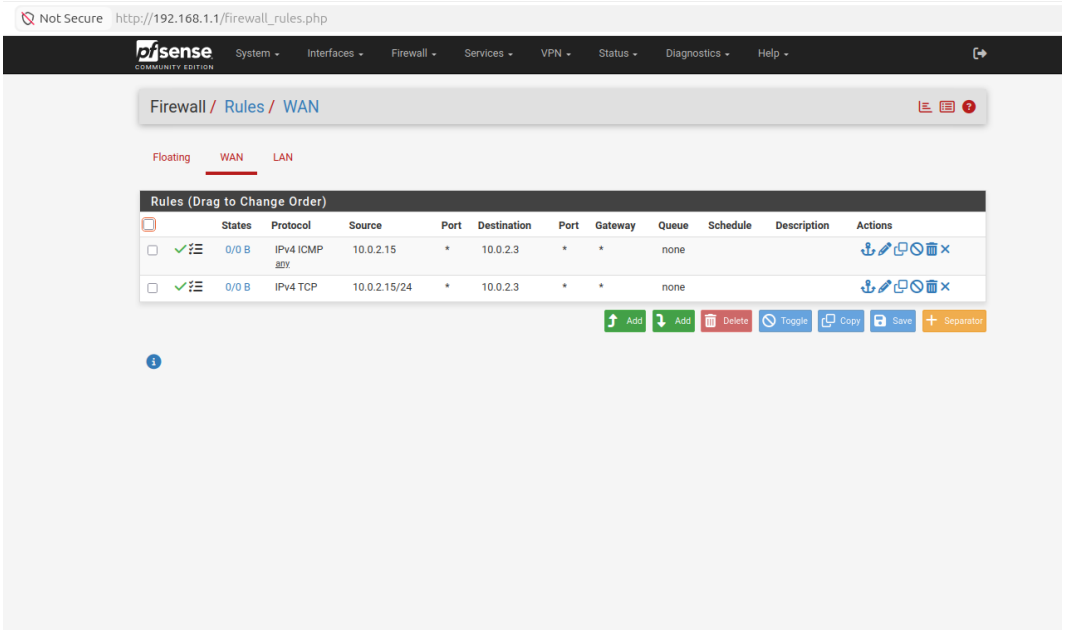


Figure 6: Both custom firewall rules applied and active

4. Simulating the SYN Flood Attack

From Kali Linux (10.0.2.15) I ran the following command:

Bash:

```
hping3 -S -p 80 --flood --rand-source 10.0.2.3
```

Before the rules were active, the attack succeeded. I captured the traffic in Wireshark and saw the flood.

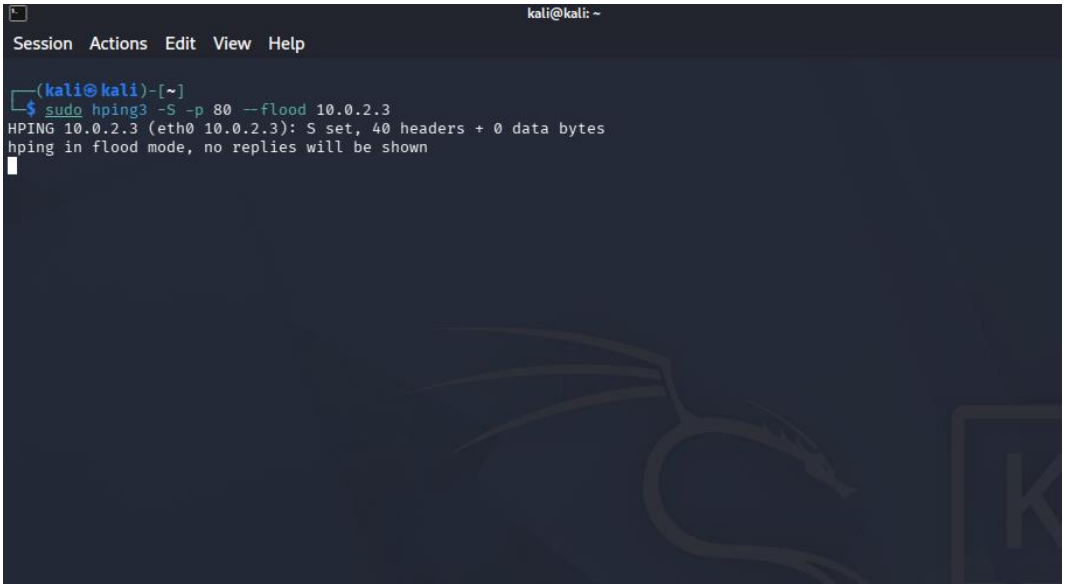


Figure 7: Launching SYN flood attack with hping3 from Kali Linux

Observing the attack in real time provided clear visibility into how quickly firewall resources can be exhausted. The hping3 command generated thousands of SYN packets per second, and Wireshark clearly showed how the firewall's resources were being consumed. I could see the web interface slowing down and becoming unresponsive, which helped me understand why SYN flood attacks are so dangerous even though they look simple to launch.

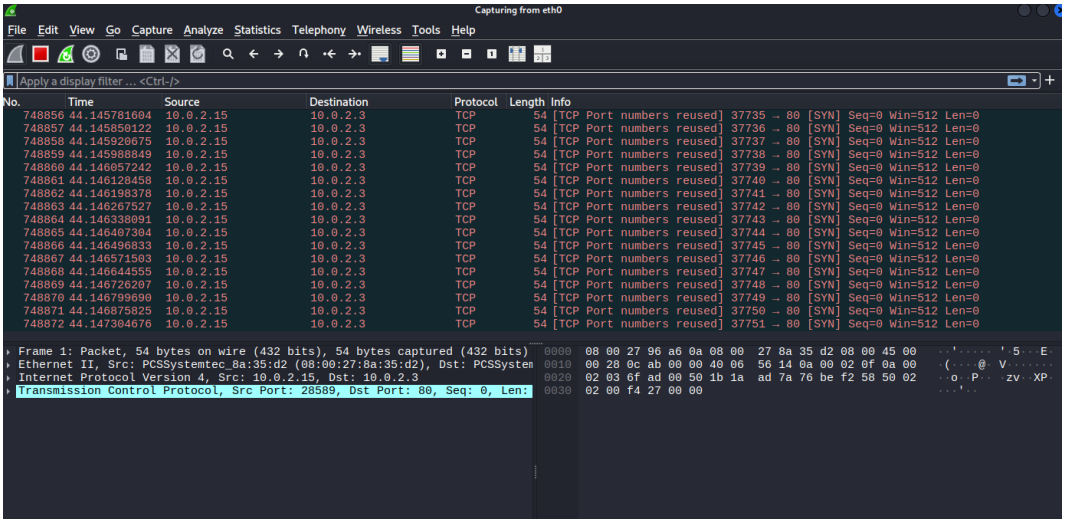


Figure 8: Wireshark capture showing successful SYN flood (before rules)

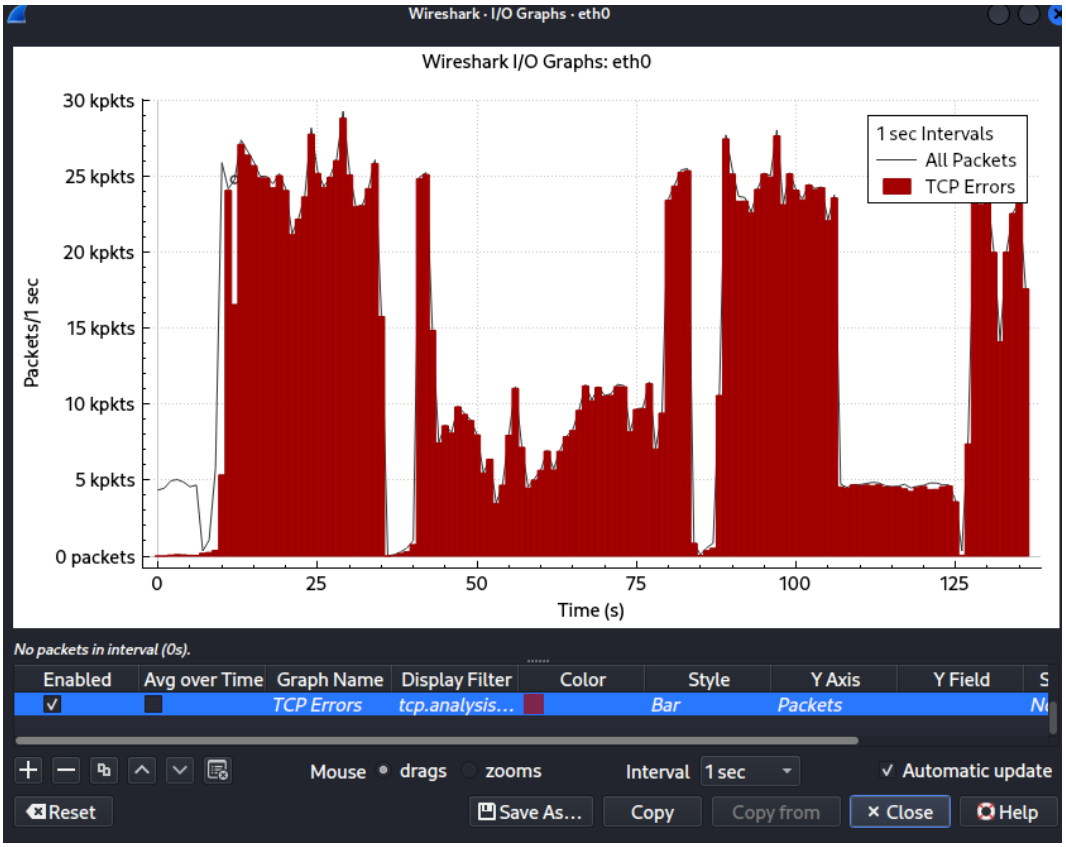


Figure 9: Wireshark I/O Graph showing successful SYN flood attack

From a detection perspective, the traffic pattern clearly deviated from normal behavior. I observed a continuous spike in SYN packets without corresponding ACK responses, indicating a potential SYN flood attack.

The repeated TCP SYN attempts from a single source IP further confirmed the malicious nature of the traffic, causing WebGUI timeout. After mitigation, attack traffic was fully dropped with no service disruption.

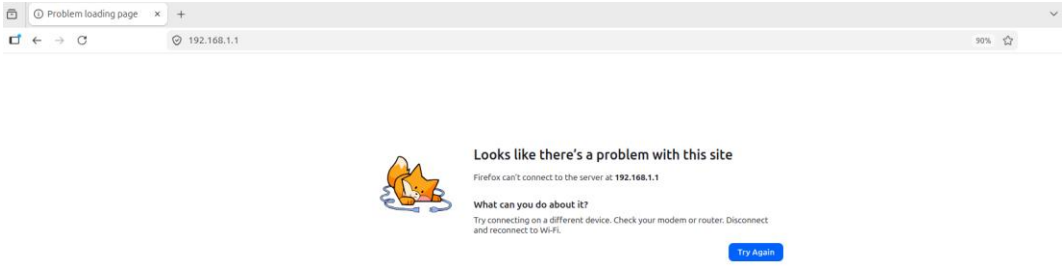


Figure 10: pfSense web interface became unresponsive during the attack

5. Blocking the Attack and Final Verification

After applying the rules, I ran the same attack again. pfSense immediately blocked the packets. ICMP requests failed, logs confirmed blocked traffic, and the dashboard reflected the dropped connections.

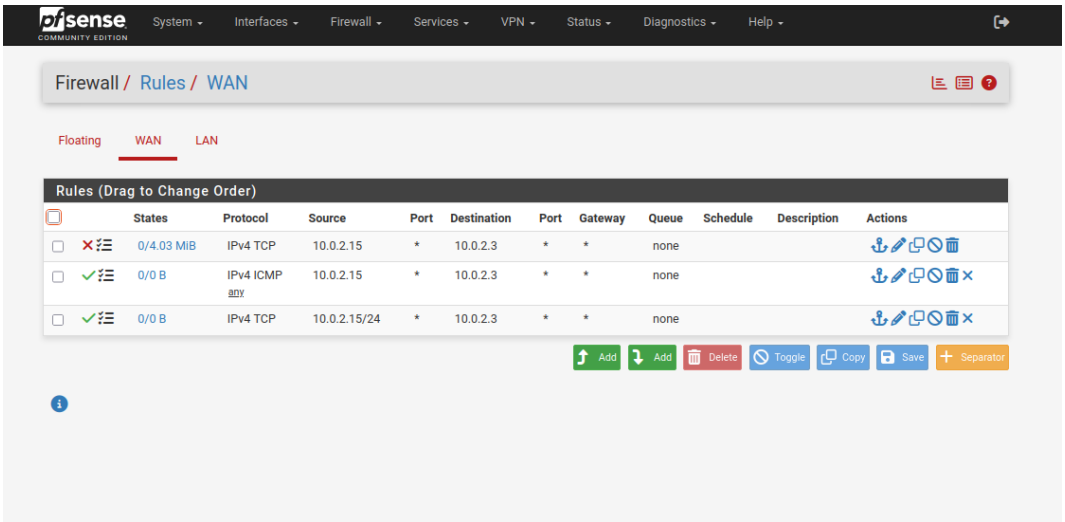


Figure 11: Firewall rules applied and active during mitigation

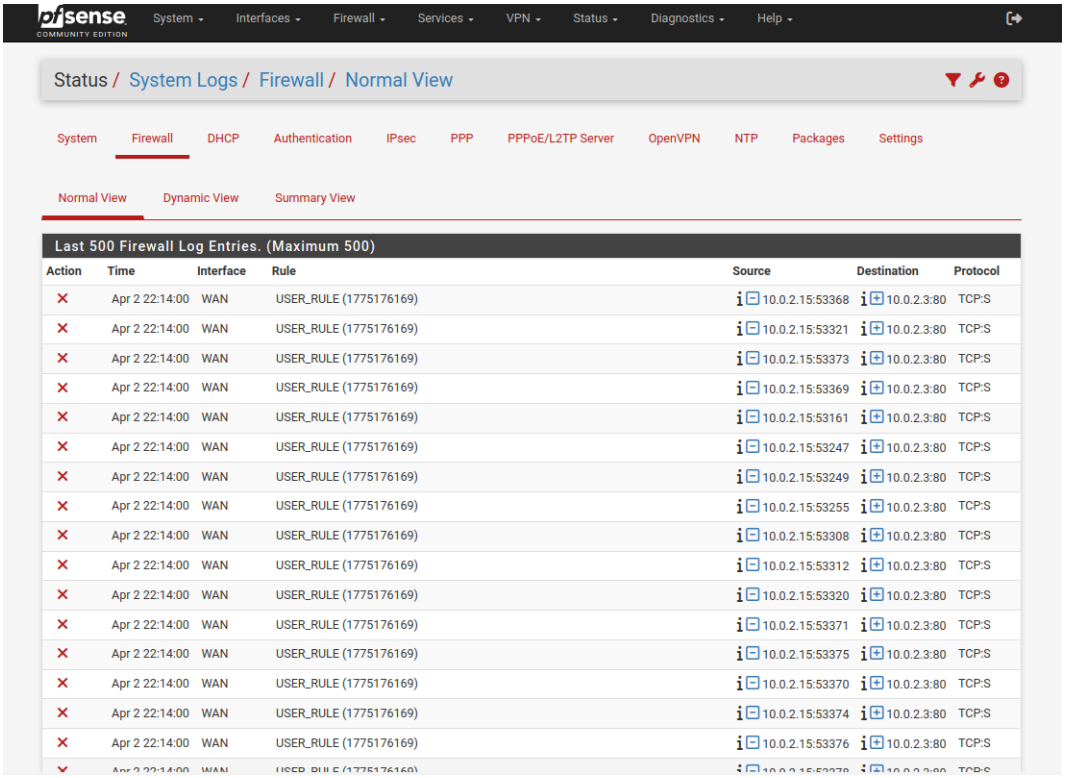


Figure 12: pfSense firewall logs showing blocked traffic

Firewall logs showed continuous packet drops, confirming the attack traffic was effectively mitigated. After the rules were applied, the same hping3 attack produced almost no effect, the packets were dropped at the firewall level before they could reach the web interface. Checking the logs and states table let me confirm exactly how many packets were blocked. This before-and-after comparison gave me strong confidence in how effective well-written firewall rules can be.

```
kali@kali: ~
Session  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::4761:3b1e:2464:447a  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:8a:35:d2  txqueuelen 1000  (Ethernet)
        RX packets 3  bytes 1240 (1.2 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 25  bytes 3214 (3.1 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 8  bytes 480 (480.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 8  bytes 480 (480.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0


┌──(kali㉿kali)-[~]
└─$ ping 10.0.2.3
PING 10.0.2.3 (10.0.2.3) 56(84) bytes of data.
```

Figure 13: Requests from Kali failed after rules were applied, confirming the traffic was successfully blocked

```
kali@kali: ~
Session  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ ping 10.0.2.3
PING 10.0.2.3 (10.0.2.3) 56(84) bytes of data.
64 bytes from 10.0.2.3: icmp_seq=1 ttl=64 time=5.51 ms
64 bytes from 10.0.2.3: icmp_seq=2 ttl=64 time=20.6 ms
64 bytes from 10.0.2.3: icmp_seq=3 ttl=64 time=19.7 ms
64 bytes from 10.0.2.3: icmp_seq=4 ttl=64 time=5.68 ms
64 bytes from 10.0.2.3: icmp_seq=5 ttl=64 time=28.2 ms
64 bytes from 10.0.2.3: icmp_seq=6 ttl=64 time=41.3 ms
^C
--- 10.0.2.3 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5614ms
rtt min/avg/max/mdev = 5.511/20.167/41.285/12.488 ms

┌──(kali㉿kali)-[~]
└─$
```

Figure 14: Ping successful again after temporarily disabling rules

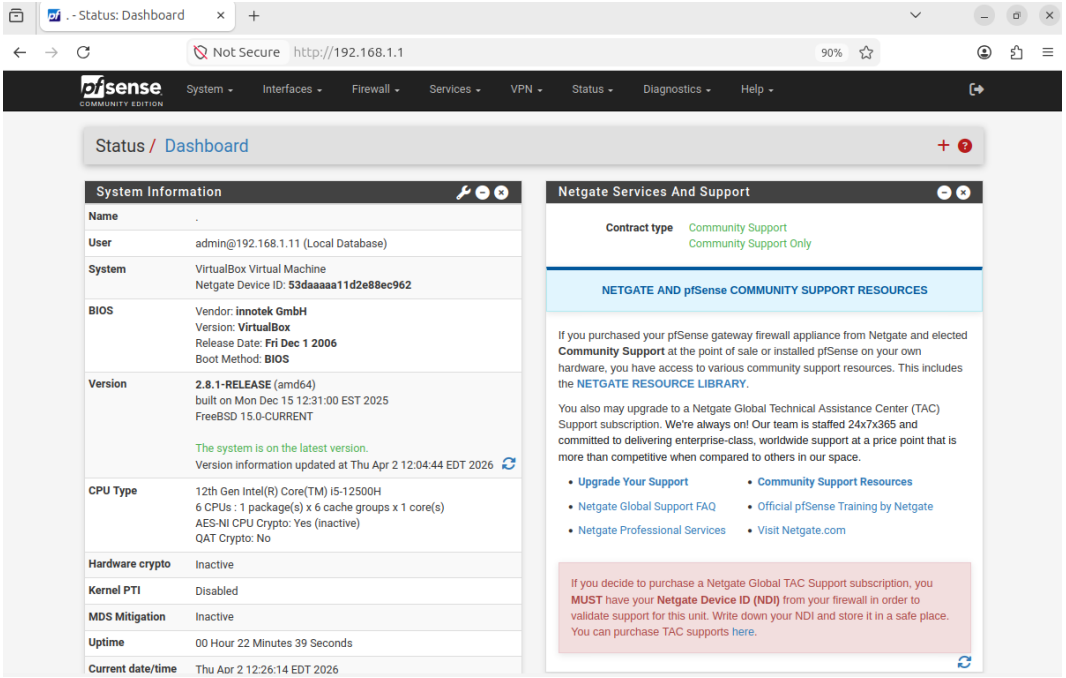


Figure 15: pfSense dashboard showing states and blocked traffic

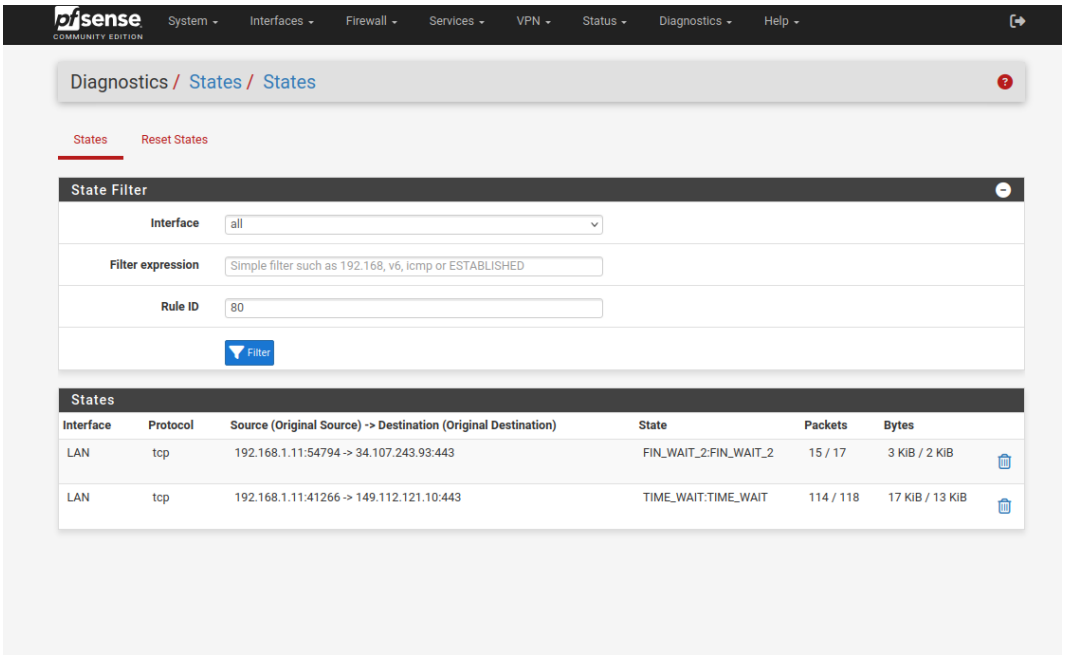


Figure 16: Detailed firewall states table during/after attack

States table confirms active connections and reduced load after mitigation.

Challenges & Troubleshooting I Faced

- Multiple attempts to reset the default password via console
- Unlocking WAN access for the private network
- On the Ubuntu machine I had to run nmcli and ip link commands several times to reset network interfaces after the flood

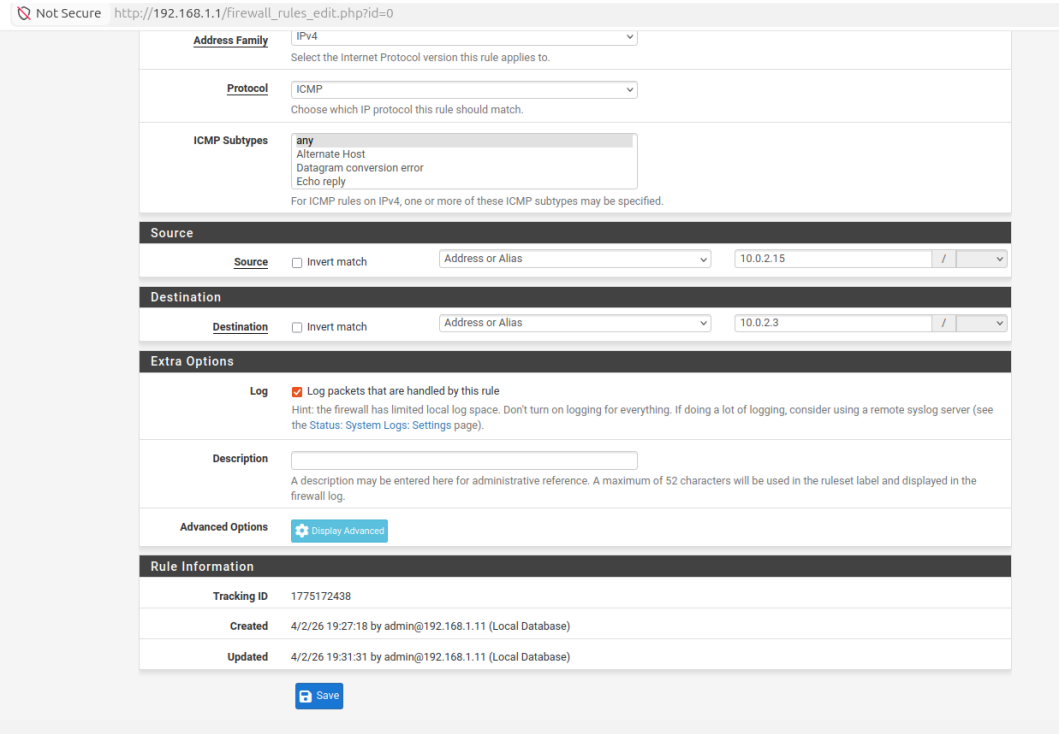


Figure 17: This highlights adjustments made to rule configuration during troubleshooting and validation

Skills Demonstrated

- Full installation and configuration of pfSense in VirtualBox
- Dual-network architecture (WAN attack side + LAN management side)
- Creating, ordering, and applying custom firewall rules
- Simulating and mitigating SYN flood (DoS) attacks with hping3
- Packet capture and analysis using Wireshark
- Log analysis and rule verification
- Network troubleshooting under attack conditions

Key Learnings & SOC Relevance

This lab gave me a much deeper understanding of how firewalls actually protect networks. I now know how to set up a secure perimeter firewall, identify and block common attacks like SYN floods, read firewall logs in real time, and test rules properly.

In a real SOC, analysts constantly monitor firewall traffic, create block rules for suspicious IPs, investigate DoS attempts, and tune rules to balance security and performance. This lab gave me direct, practical experience with all of those responsibilities.

In a production environment, additional layers of defense would be implemented to further strengthen protection against DoS attacks:

• Enable SYN flood protection mechanisms such as SYN cookies and rate limiting
• Deploy IDS/IPS solutions (Snort or Suricata) within pfSense for deeper traffic inspection
• Forward firewall logs to a SIEM (e.g., Splunk) for centralized monitoring and alerting
• Implement Geo-IP filtering and threat intelligence-based blocking
• Configure alerting thresholds to detect abnormal traffic spikes in real time

These enhancements would improve detection, response time, and overall network resilience.

Resources Used

- Video tutorial: "pfSense Firewall Lab" by The Social Dork - https://youtu.be/-yRvfbEIT7M
- pfSense official documentation
- hping3 and Wireshark for attack simulation and analysis
- Full screenshots: GitHub